

Bachelor Projekt – MUNCHORA



[Home](#) [Generate Recipe](#) [Grocery Lists](#) [FAQ](#) [About](#) [Admin](#)



Discover Your Next Culinary Adventure

Let AI craft personalized recipes from cuisines around the world. Organize your cooking with smart grocery lists and share culinary discoveries with friends.

[Start Cooking](#) →

10K+
Recipes Created

5K+
Happy Cooks

★★★★★ 4.9/5 rating



Everything You Need to Cook Better

From AI-generated recipes to smart grocery planning

Holdbetegnelse

SOFT-GBG-F25A

Site

<https://munchora.pro>

Alexander Christensen

<https://github.com/realkoder>

Github repo

<https://github.com/realkoder/bachelor-project-munchora>

Dato for aflevering

08/06-2026

Indholdsfortegnelse

1.0 Indledning	4
1.1 Problemformulering	4
2.0 Udviklingsproces	5
2.1 Kravsspecifikation	5
2.1.1 Funktionelle krav	6
2.1.2 Ikke funktionelle	7
2.1.3 Sammenhæng mellem kravspecifikation og test	8
2.2 Agil udvikling	10
3.0 Systemets arkitektur	11
3.1 Microservices	12
3.1.1 Frontend	13
3.1.2 Backend Services	13
3.1.3 DLS Patterns	18
3.1.4 Databases	20
3.2 Environments	22
3.2.1 Udviklingsmiljø	22
3.2.2 Monitorering	24
4.0 Test	25
4.1 Unit Tests	26
4.2 Integration og API Tests	29
4.3 End to end tests	30
4.4 Continuous Integration (CI)	31
5.0 Konklusion	32
6.0 Litteraturliste	34
7.0 Bilag	36
7.1 DB Physical Entity Diagrams	36

7.1.1 ai_db	36
7.1.2 auth_db	37
7.1.3 recipes_db.....	38
7.1.4 shopping_lists_db.....	39

1.0 Indledning

Til mit afsluttende projekt på professionsbacheloren Software Udvikling tager jeg udgangspunkt i udviklingen af applikationen Munchora. Projektet har til formål at anvende centrale teknologier, metoder og principper, der er blevet gennemgået i løbet af uddannelsen.

Munchora er en webbaseret applikation, der henvender sig til private slutbrugere, der ønsker inspiration til madlavning samt værktøjer til at strukturere og dele daglige indkøb og opskrifter. Applikationen giver brugerne mulighed for at oprette sig enten manuelt eller via Google Single Sign-On, hvorefter man kan tilgå og dele opskrifter med andre brugere. Systemet er integreret med kunstig intelligens (OpenAI - gpt-4.1-mini) til generering af opskrifter for en mere dynamisk og personlig brugeroplevelse. Brugeren kan gennem prompts generere opskrifter ud fra specifikke ønsker såsom kostpræferencer, bestemte ingredienser eller et ønsket køkken eksempelvis italiensk, asiatisk eller vegetarisk mad. Det er også muligt at oprette og dele indkøbslister mellem flere brugere, hvilket understøtter samarbejde omkring planlægning og håndtering af indkøb. Flere brugere kan få adgang til den samme indkøbsliste og tilføje, opdatere eller markere varer som købt, hvilket gør koordinering af fælles indkøb mere effektiv og overskueligt. Jeg har fokuseret på at udvikle en skalerbar og brugervenlig applikation samt integrere moderne teknologier, herunder webudvikling, autentifikation og anvendelse af kunstig intelligens. Denne rapport vil også forsøge at reflektere over designvalg, arkitektur og brugeroplevelse i udviklingen af moderne softwareløsninger.

1.1 Problemformulering

Hvordan udvikles en robust og skalerbar webapplikation til generering af opskrifter og håndtering af indkøbslister ved brug af en microservice arkitektur, så systemet forbliver fejlforsistent under høj belastning?

Med fokus på udviklingen af en skalerbar platform vil jeg undersøge, hvordan de enkelte services bedst muligt kommunikerer med hinanden og hvordan uafhængig skalering og

deployment optimerer systemets pålidelighed for at understøtte en brugervenlig oplevelse.

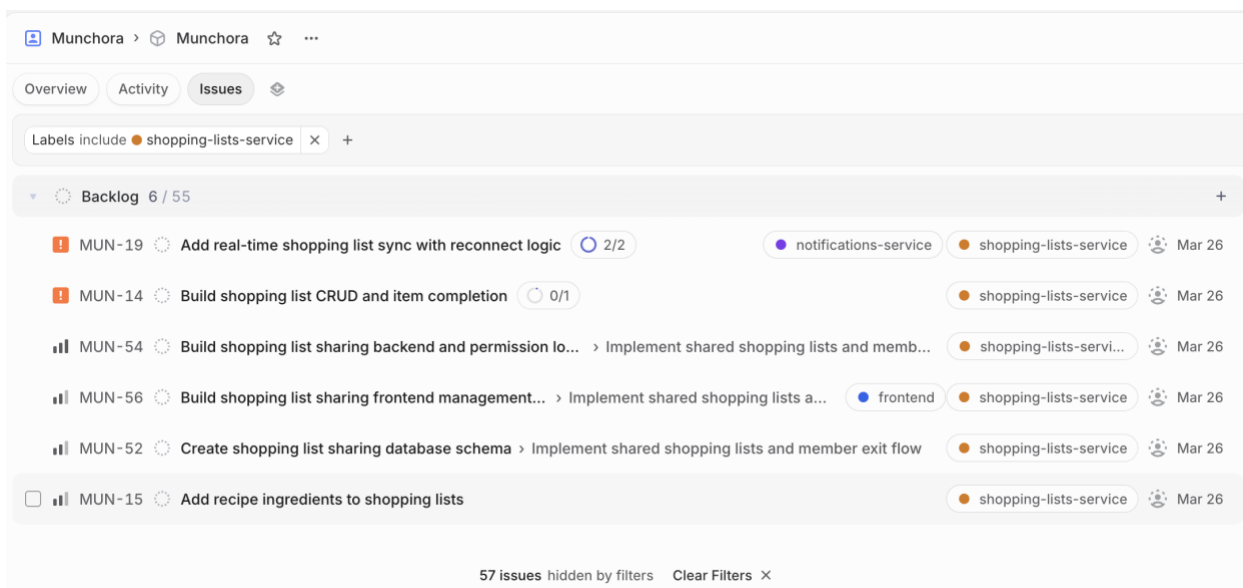
Platformen skal understøtte funktioner som brugerhåndtering, søgning af opskrifter, generering og deling af indkøbslister. Systemet skal være pålideligt og potentielt være i stand til at håndtere flere tusinde samtidige brugere. Arkitekturen skal sikre applikationens robusthed, skalerbarhed og fejltolerance, hvor services kan udvikles, deployes og skales uafhængigt.

2.0 Udviklingsproces

I dette afsnit vil jeg beskrive min udviklingsproces og de overvejelser, der ligger bag valget af min arbejdsmetode for udviklingen af Munchora. Jeg giver et overblik over, hvordan udviklingen er struktureret i praksis, samt hvilke værktøjer jeg har anvendt til planlægning og opgavestyring. Jeg vil også komme ind på, hvordan min proces har været væsentligt anderledes sammenlignet med processen for et team baseret setup.

2.1 Kravsspecifikation

Kravene er groft organiseret og indsamlet i Linear, hvilket har fungeret som et centralt backlog værktøj. Her er opgaver og forbedringer blevet registreret som issues (se figur 1), hvilket har givet mig et kontinuerligt overblik over Munchoras udvikling og sikret en iterativ proces.



Figur 1 – Overblik over issues defineret for shopping-list-service

Jeg har gjort brug af FURPS-modellen (Noble Prog 2017) til opsætningen af kravene, som efterfølgende er defineret i Linear med nødvendige sub-issues og user stories. De funktionelle krav er primært formuleret med udgangspunkt i konkrete brugerinteraktioner, mens de ikke-funktionelle krav i højere grad afspejler systemets overordnede kvaliteter og den samlede brugeroplevelse.

2.1.1 Funktionelle krav

De funktionelle krav for Munchora præsenteres her på et overordnet niveau som centrale features eller større funktionelle enheder. Disse repræsenterer systemets primære egenskaber set fra et bruger- og forretningsperspektiv. Hver feature kan nedbrydes efterfølgende i mindre, konkrete opgaver og user stories (figur 2), som gør det muligt at planlægge, implementere og validere funktionaliteten iterativt. Denne opdeling sikrer, at komplekse funktioner bliver håndterbare og kan udvikles trinvis, samtidig med at der bevares en klar kobling mellem overordnede krav og den faktiske implementering.

2.1.1.1 Brugerregistrering

- Brugere skal kunne oprette sig enten manuelt eller igennem Google SSO.

- GDPR-compliance og mulighed for at slette sin profil og tilhørende data eller anonymisering
- Administrering af ens brugerprofil såsom ændring af navn, e-mail og beskrivelse

2.1.1.2 Opskrifter

- Prompt baseret generering af opskrifter, som efterfølgende kan slettes og redigeres enten med yderligere prompting eller manuelle ændringer.
- Tilføjelse af billede til en opskrift.
- Deling af opskrifter ved at gøre adgangen til opskriften offentlig.
- Søgefunktionalitet for opskrifter; enten brugerens egne eller delte opskrifter fra andre brugere.

2.1.1.3 Indkøbslister

- Brugere skal kunne oprette, redigere og slette en indkøbsliste.
- Tilføjelse af varer til en given indkøbsliste fra ingredienslisten for en valgt opskrift.
- Dele indkøbsliste med andre brugere og herefter også at kunne fjerne andre brugere fra ens egne indkøbslister.

2.1.2 Ikke funktionelle

2.1.2.1 Reliability Requirements

- Brugervenlige succes og fejlbeskeder ved brug af applikationen.
- Sikre høj tilgængelighed med minimal nedetid ideelt 99,0% opetid.

2.1.2.2 Performance Requirements

- Hver side må have en indlæsnings tid på max 2 sekunder under normale omstændigheder.
- Applikationen skal kunne håndtere 1.000 samtidige brugere uden påvirkning af applikationens opførsel.

- Arkitekturen skal gøre det muligt at skalere systemet afhængigt af brugermængden.

2.1.2.3 Supportability Requirements

- Q&A side for at sikre en god brugeroplevelse.
- Mulighed for, at brugerne kan give feedback eller på anden måde kundesupport.
- Implementering af monitorering og sporing af systemet opførsel.

2.1.3 Sammenhæng mellem kravspecifikation og test

Munchoras kernefunktionalitet består af AI-baseret generering af opskrifter samt mulighed for ændring, administrering og deling af opskrifter og indkøbslister mellem brugere. Disse funktioner understøtter både individuel brug samt deling og samarbejde systemets brugere imellem. For at kunne tilgå disse funktioner er det en nødvendig forudsætning, at brugeren oprettes og derefter kan autoriseres. Når en bruger er logget ind, gives der adgang til systemets funktionalitet baseret på vedkommendes rettigheder. Samtidig er sikker håndtering af brugerdata et centralt krav. Adgang til data på tværs af brugere er strengt kontrolleret og forudsætter korrekt autorisation, hvilket stiller krav til en veldefineret arkitektur, hvor autentifikation, autorisation og datatilgange håndteres konsistent og sikkert. User stories med tilhørende acceptance criteria (figur 2) kan anvendes for at sikre en klar sammenhæng mellem kravspecifikationen og test som fundament for udviklingen. Disse krav kan omsættes direkte til testscenarier gennem anvendelse af Behavior-Driven Development (BDD) (Cucumber u.d.), hvor funktionaliteten beskrives og verificeres ud fra konkrete brugerhandlinger og forventede resultater (figur 3).

User stories with acceptance criteria for Shopping Lists

Story: Create, update and delete shopping lists

As a user, I want to create, update and remove shopping lists so that I can organize my shopping activity.

Acceptance criteria

A user can create a shopping list with the required fields.

A user can update the name for a shopping list.

A user can delete a shopping list they are allowed to manage.

Story: Manage shopping list items

As a user I want to add, update, remove, and complete shopping list items so that I can track what I need to buy.

Acceptance criteria

A user can add a new item to a shopping list.

A user can update item details.

A user can mark an item complete or incomplete.

A user can remove an item.

Story: Share a shopping list

As a user I want to share a shopping list with other users so that we can collaborate.

Acceptance criteria

A user can invite or grant access to another user using the supported sharing flow.

Shared users can access the list according to their granted permissions.

Story: Stop sharing a shopping list

As a user I want to revoke shared access to a shopping list so that I can control collaboration.

Acceptance criteria

An authorized user can remove another user's access.

Removed users lose access promptly.

The list membership view reflects the updated access state.

Story: Leave a shared shopping list

As a collaborator I want to leave a shared shopping list so that I can remove it from my workspace.

Acceptance criteria

A collaborator can leave a shared shopping list they do not own.

Remaining collaborators retain their access.

Figur 2 – user stories med acceptance criteria for shopping list domænet

Dette kan skabe en tæt kobling mellem krav, implementering og validering. Når kravene er klart definerede og testbare, reduceres risikoen for misforståelser i udviklingsprocessen og bidrager til en højere kvalitet af det endelige system.

```
# munchora/backend/shopping-lists-service/spec/requests/api/v1/shopping-lists/manage_lists_spec.rb
RSpec.describe 'Manage shopping lists', type: :request do
  # omitted code
  describe 'As a user I want to manage my shopping lists' do
    describe 'creating a shopping list' do
      it 'creates a list with valid data' do
        # omitted code
      end

      it 'fails without required fields' do
        # omitted code
      end

      it 'requires authentication' do
        # omitted code
      end
    end

    describe 'updating a shopping list' do
      # omitted code

      it 'successfully with valid data' do
        # omitted code
      end

      it 'fails without required fields' do
        # omitted code
      end

      it 'requires authentication' do
        # omitted code
      end
    end

    describe 'deleting a shopping list' do
      # omitted code
      it 'allows owner to delete' do
        # omitted code
      end

      it 'removes list from index' do
        # omitted code
      end

      it 'rejects non owners from deleting list but will unshare' do
        # omitted code
      end

      it 'prevents unauthorized deletion' do
        # omitted code
      end
    end
  end
end
```

Figur 3 - Her ses skelettet for tilhørende tests for administrering af shopping list funktionaliteten

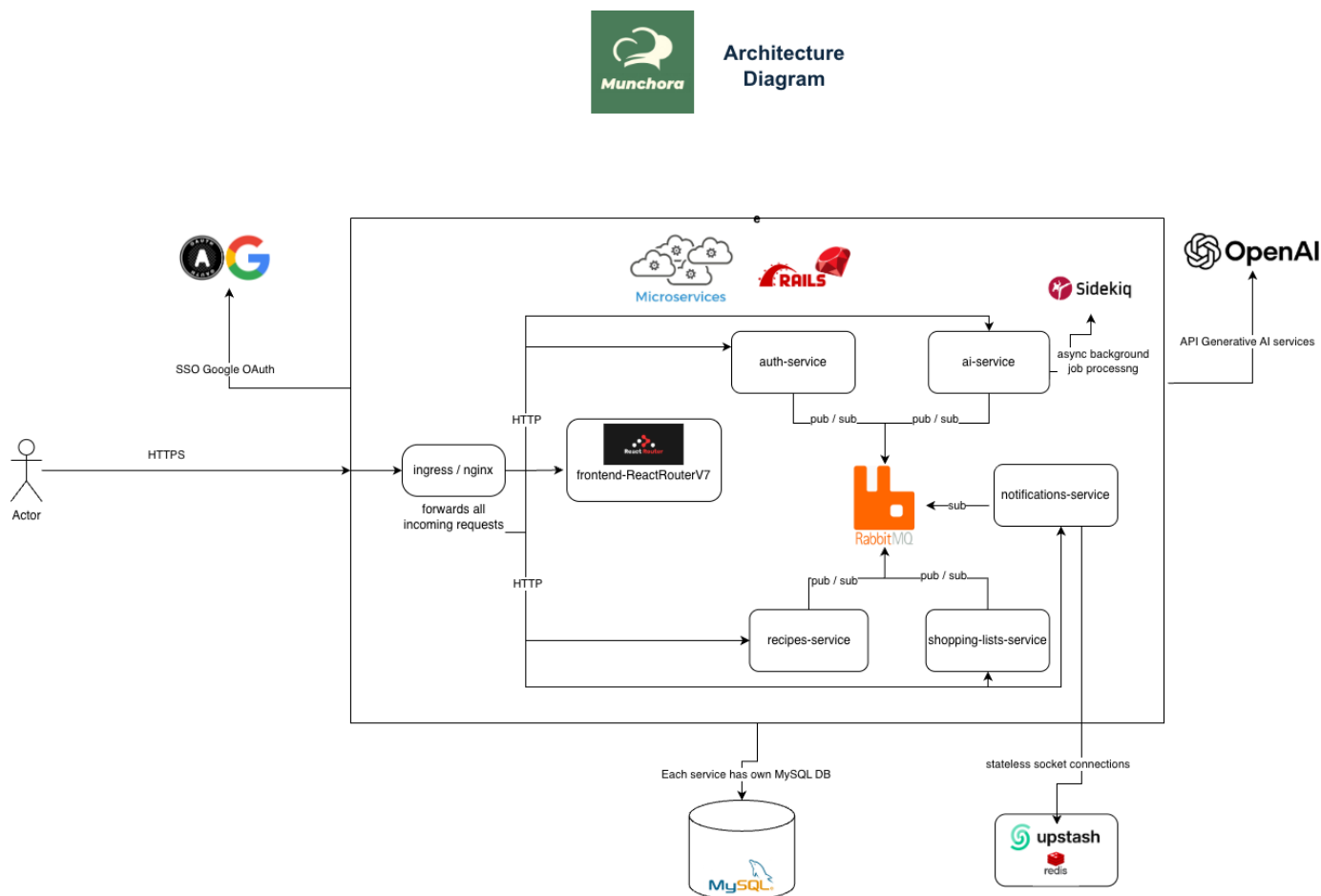
2.2 Agil udvikling

Jeg har arbejdet alene med Munchora, og jeg har derfor ikke fulgt en stramt veldefineret udviklingsproces. Der har ikke været behov for koordinering mellem flere udviklere, hvortil et formelt framework så som Scrum eller Extreme Programming kunne have været inddraget til at sikre en sund udviklingsproces. Min proces er i stedet blevet tilpasset efter

en mere fleksibel og iterativ tilgang, hvor jeg løbende har arbejdet med implementering af features, fejlrettelser og forbedringer baseret på et kontinuerligt overblik over systemets tilstand. Denne tilgang har gjort det muligt for mig hurtigt at skifte fokus mellem forskellige dele af systemet og reagere effektivt på opståede problemer uden faste ceremonier eller sprint-strukturer. Samtidig har det givet mig en høj grad af kontrol over prioritering og fremdrift i projektet. Hvis jeg havde udviklet projektet i samarbejde med andre, ville en mere struktureret udviklingsproces have været nødvendig. I så fald ville jeg have inddraget elementer fra Scrum, da det er et af de mest udbredte frameworks i softwareindustrien, og derfor noget mange udviklere forventes at have erfaring med. Scrum dikterer en opdeling af sprints, prioritering af opgaver i en backlog samt afholdelse af sprint planning, review og retrospektiv for at sikre fælles retning og løbende forbedring (Radigan u.d.). Til trods for min fleksible tilgang har jeg anvendt værktøjet Linear til at skabe overblik over opgaver og features. Her er tasks blevet defineret og organiseret, hvilket har fungeret som støtte til planlægning og strukturering af mit arbejde. Værktøjet har med sin AI understøttelse, bidraget til at effektivisere min håndtering af opgaver og skabt et kontinuerligt overblik over min fremdrift.

3.0 Systemets arkitektur

Følgende afsnit vil redegøre for den overordnede arkitektur og de væsentligste designvalg for systemets komponenter, deres strukturer og hvordan services gør brug af hinanden. Applikationen er bygget op med en klar opdeling mellem en frontend og flere backend services samt datalagring for at understøtte en skalerbar og vedligeholdelsesvenlig kodebase. Arkitekturen er baseret på moderne webteknologier og principper, der muliggør fleksibel udvikling og integration med eksterne services. Jeg har lagt vægt på løs kobling mellem systemets dele samt en hensigtsmæssig fordeling af ansvar mellem klient og server. Klienten/frontenden er primært ansvarlig for præsentation af data, navigation, lokal state management og brugerinteraktioner, mens serveren håndterer forretningslogik, autorisation, datavalidering og persistens. De følgende underafsnit uddyber de enkelte komponenter og deres konkrete implementering.



Figur 4 – Overordnede arkitektur design for Munchora

3.1 Microservices

Systemet er opdelt i mindre selvstændige services med ansvar for hvert deres domæne. Formålet er at sikre en fleksibel og skalerbar arkitektur gennem tydelig ansvarsfordeling. Som Martin Fowler påpeger, er den største frustration ved monolitter, at selv små ændringer kræver, at hele systemet bygges og deployes igen (Fowler 2014). Med microservices kan en enkelt service ændres, deployes og skales uafhængigt, hvilket tillader mere agilitet og mulighed for at eksperimentere uden at risikere systemets stabilitet. Dette koncept kalder Martin Fowler for evolutionær design, hvilket jeg forstår som, at systemet kan udvikle sig trinvist som behovene ændres, ligesom naturens evolutionære processer tillader arter at overleve ved at tilpasse sig én komponent ad gangen frem for at omskrive hele organismen.

3.1.1 Frontend

Frontenden er skrevet i React.js og anvender frameworket ReactRouter version 7 (RRv7). Opsætningen inkluderer integration af TypeScript for typesikkerhed samt TailwindCSS til effektiv og responsiv styling og en allerede konfigureret Dockerfile til containerisering for at opnå konsistens på tværs af udviklings- og produktionsmiljøer. Valget af RRv7 frem for en mere manuel opsætning med eksempelvis Vite er truffet på baggrund af disse indbyggede konfigurationer, som reducerer kompleksiteten og forsimples opsætningen. En væsentlig fordel ved RRv7 er understøttelsen af både server side rendering og client side rendering. Ved at renderer statiske sider og ikke autoriserede sektioner på serveren reduceres belastningen på brugerens enhed, hvilket resulterer i hurtigere indlæsningstider og en mere flydende brugeroplevelse.

Til håndtering af global state har jeg anvendt Jotai, der gennem atoms muliggør en simpel tilgang til delt tilstand på tværs af React components. Derudover benyttes TanStack React Query til datahåndtering og caching på klientens side. Dette sikrer at tidligere anmodet data genbruges ved efterfølgende forespørgsler, indtil det betragtes som forældet, hvilket er sat til 5 minutter, for at reducere unødvendige netværkskald og det forbedrer applikationens performance.

3.1.2 Backend Services

Dette afsnit beskriver de forskellige backend services til Munchora med fokus på deres ansvar, opbygning og indbyrdes samspil. De forskellige backends er udviklet i Ruby med Ruby on Rails som framework og fungerer udelukkende som et API, der betjener frontenden, for at sikre en klar adskillelse mellem forretningslogik og præsentationslaget. Jeg har valgt Ruby on Rails på grund af Rubys læsbare og fleksible syntaks, som understøtter hurtig og overskuelig udvikling. Samtidig tilbyder Rails et modent økosystem med stærk community support, et rigt udvalg af biblioteker (Gems) samt effektive CLI-værktøjer, der gør det hurtigt at generere og strukturere centrale dele af applikationen (Rails, Rails Guides 2026). Et centralt mønster for de forskellige Rails services er brugen af service objekter, hvor controllers så vidt som muligt implementeres med små funktioner

og i stedet delegerer forretningslogik til separate klasser, eksempelvis ved følgende kald, der opretter en flygtig instans af `UserSearchService` under `Users` modulet og herefter eksekveres `call` metoden:

```
Users::UserSearchService.new(params).call
```

Dette kan være med til at gøre koden mere vedligeholdelsesvenlig for fremtiden, da koden bliver mere modulær og lettere at forstå. Det forøger også kodens testbarhed, da forretningslogik kan testes isoleret uden at involvere controllers. At der oprettes en ny instans for hver forespørgsel, er typisk for Ruby (Kosmowski 2017) og bidrager til at undgå en delt mutable tilstand, hvilket kan reducere risikoen for fejl og uforudsigelig adfærd. Sammenlignet med frameworks som Spring Boot anvendes der en anden tilgang, hvor services ofte defineres som singleton beans (Strmecki 2026), der genbruges på tværs af controllers via dependency injection. Dette kan forbedre ydeevnen ved at minimere objektoprettelse og giver en centraliseret styring af afhængigheder, men til gengæld kræver det større disciplin i forhold til håndtering af delt tilstand og systemet kan hurtigere blive mere komplekst at konfigurere og forstå. Efter at have arbejdet med både Rails og Spring Boot foretrækker jeg personligt Rails til denne type projekt. Rails giver mulighed for hurtigere udvikling gennem en mere simpel syntaks og et stort økosystem af gems, som reducerer mængden af boilerplate kode betydeligt.

Til datapersistens og håndtering af forretningslogik anvendes ActiveRecord, som er Rails indbyggede Object Relational Mapping (ORM). Dette muliggør en deklarativ definition af datamodeller og deres indbyrdes relationer gennem migrationer, som sikrer en konsistent og versioneret database struktur. For at optimere datatransmissionen og applikationens ydeevne er der implementeret paginering via Gem Kaminari. Dette begrænser mængden af data i de enkelte API responser og reducerer dermed belastningen på både server og klient. De følgende underafsnit uddyber de enkelte services og deres rolle i arkitekturen.

Auth-service

Oprettelse og ændringer af brugere håndteres af denne service, og derfor også Munchoras "single source of truth" for autentificering og autorisation af brugere. Det er udelukkende denne service, der er ansvarlig for at udstede JWT tokens med en private pem file, som de øvrige services benytter til at validere med public pem file, om en bruger

er autentificeret og har adgang til givne ressourcer. Servicen håndterer også login via Google single sign on (SSO) samt oprettelse og vedligeholdelse af brugere. Ved succesfuld autentificering genererer auth-service en JWT-token, som indeholder de nødvendige oplysninger til identifikation og autorisation, og som efterfølgende anvendes af andre services til at sikre en konsistent og sikker adgangskontrol på tværs af systemet.

Recipes-service

Recipes-service er ansvarlig for håndtering af opskrifter i systemet, herunder oprettelse ved hjælp af AI eller manuelle ændringer og sletning. Servicen kan igangsætte generering af opskrifter ved at gøre brug af ai-service, som består af en asynkron kommunikation gennem RabbitMQ. Når ai-service har genereret en opskrift, sendes opskriften tilbage til recipes-service, som herefter validerer og persister opskriften i databasen. Herefter vil et event igangsættes til notifications-service, som endeligt vil informere brugeren om den færdige opskrift.

Recipes-service anvender de udstedte JWT tokens fra auth-service til at håndhæve adgangskontrollen. Servicen har adgang til public pem fil (public key), som anvendes til at verificere, at den udstedte token er gyldig og generet af auth-service.

Adgangen til opskrifter er opdelt således at private opskrifter udelukkende er tilgængelige for forfatteren (den bruger, der har genereret opskriften), mens offentlige opskrifter er tilgængelige for alle systemets brugere uden yderligere restriktioner.

Ai-service

Ai- service er ansvarlig for integrationen med OpenAI og håndteringen af brugernes genereringer af opskrifter. Servicen er designet til at være isoleret fra øvrig forretningslogik, således AI relaterede funktioner kan udvikles og skales uafhængigt. Generering af opskrifter foregår asynkront ved brug af Sidekiq til jobprocessering. Dette flow igangsættes ved at recipe-service emitter et event via RabbitMQ til kanalen "ai_prompt.requests", som ai-service abonnerer på. Et sådan event håndteres med Sidekiq til jobprocessering, hvilket sikrer, at de potentielt tidstunge kald til OpenAI håndteres i baggrunden uden at blokere for øvrige processer. Denne asynkrone tilgang sikrer en robust og skalerbar håndtering af AI opgaver, selv under høj belastning. Det

sikrer, at ressourcekrævende AI forespørgsler ikke blokerer systemets øvrige funktionalitet. Sidekiq er integreret med Redis, hvor jeg anvender en cloudbaseret løsning (Upstash), som fungerer som kø- og state management for jobs. Redis spiller en central rolle i forhold til skalerbarhed, da det sikrer, at jobs håndteres korrekt på tværs af flere Sidekiq-processer, hvor det fjerner staten fra hver enkelt service, men sikrer en "single source of truth". Dette forhindrer, at det samme job eksekveres flere gange, selv i scenarier hvor flere instanser af AI-service kører parallelt. Når en opskrift er genereret, sendes resultatet tilbage til recipe-service igennem RabbitMQ, som herefter står for videre behandling og persistens.

Shopping-lists-service

Håndteringen og ansvaret for brugernes indkøbslister i Munchora varetages af shopping-list-service. Det er muligt for brugerne at oprette, slette, tilføje varer til og dele indkøbslister. Servicen håndterer relationer mellem brugere og indkøbslister, herunder rettigheder til at se og redigere lister, hvor brugerne valideres på samme måde som tidligere beskrevet med JWT-token. For at sikre løs kobling mellem auth-service er brugerdata ikke direkte delt på tværs af databaser. Brugere fra auth-service er i stedet "kopieret" ind i shopping-list-service som en lokal model `ShoppingListOwner`, hvilket også er tilfældet i recipes-service. Det betyder, at hver service har sin egen repræsentation af en bruger med de nødvendige attributter til det specifikke domæne. Koblingen til auth-service opretholdes via en reference (`auth_user_id`), som gør det muligt at identificere brugeren på tværs af services uden at skabe en direkte afhængighed til auth-service databasen. Denne tilgang følger principperne i en microservice arkitektur, hvor hver service ejer sin egen database og datamodel. Ved at duplikere og afgrænse brugerdata således opnår man en højere grad af robusthed og uafhængighed mellem services, da ændringer i auth-service ikke direkte påvirker shopping-list-service. Samtidig gør det det muligt for hver service at skalere og udvikle sig uafhængigt.

Autorisation af brugerne sikrer, hvilken adgang hver bruger har til specifikke indkøbslister hvad end de ejer listen eller har delt adgang tilhørende en anden brugers liste. Derudover kan ændringer i indkøbslister, såsom tilføjelse eller opdatering af varer, emitte RabbitMQ

events gennem kanalen "notification-service" til notifications-service, så involverede brugere modtager opdateringer i realtid.

Notifications-service

Dette er fyrtårnet for Munchora, som står for at udstede notifikationer fra de forskellige services til brugerne. Notifications-service modtager hændelser fra de øvrige services igennem en event baseret arkitektur opsat med RabbitMQ. Disse hændelser kan eksempelvis være fra recipe-service, når en opskrift er færdiggenereret, eller shopping-list-service, når der sker ændringer i en indkøbsliste. Hændelserne behandles asynkront og omsættes til notifikationer rettet mod specifikke brugere. Kommunikationen til brugerne sker ved hjælp af en WebSocket forbindelser, der er implementeret med ActionCable (Rails, Action Cable Overview u.d.), hvilket muliggør realtidsopdateringer i applikationen. Hver klient opretter forbindelse til en specifik instans af notifications-service, hvor forbindelsen håndteres lokalt og vil derfor være stateful, da den enkelte instans holder styr på sine aktive brugere. For at understøtte skalering på tværs af flere instanser er ActionCable konfigureret med Redis, hvor jeg her har valgt at bruge Upstash, som er en cloud baseret Redis udbyder. Redis anvendes som en fælles pub/sub mekanisme og key value database mellem instanserne. Når en service eksempelvis recipe-service udsender et event igennem RabbitMQ kan det håndteres af en hvilken som helst instans af notifications-service, som så publicerer dette event til Redis, hvorefter de øvrige notifications-service instanser modtager beskeden. Hver instans vurderer derefter lokalt, om den har en aktiv forbindelse til den pågældende bruger, og kun den relevante instans videresender notifikationen til brugeren. Det betyder, at det ikke er nødvendigt at route events til en konkret instans af notifications-service. I stedet kan enhver instans, der modtager en hændelse fra RabbitMQ, udsende notifikationen via ActionCable, hvorefter Redis sikrer, at beskeden distribueres korrekt til den instans, der har den aktive WebSocket forbindelse. Et forsimplet eksempel på denne mekanisme kan illustreres således:

```
# munchora/backend/notifications-service/lib/rabbitmq_consumer.rb
class RabbitmqConsumer
  def self.start
    NOTIFICATION_SERVICE_QUEUE.subscribe(block: true, manual_ack: true) do |delivery_info, properties,
payload|
      begin
        utf8_payload = payload.force_encoding('UTF-8')
        parsed_payload = JSON.parse(utf8_payload)
        payload = OpenStruct.new(parsed_payload)

        payload.receivers.each do |hashed_user|
          user = OpenStruct.new(hashed_user)
          user_id = user.auth_user_id
          ActionCable.server.broadcast("notifications:#{user_id}", {
            type: payload.type,
            payload: payload.payload,
            created_at: payload.created_at
          })
        end

        # Acknowledge the message
        RABBITMQ_CHANNEL.ack(delivery_info.delivery_tag)
      end
    end
  end
end
```

Og i den tilhørende channel:

```
# munchora/backend/notifications-service/app/channels/notifications_channel.rb
class NotificationsChannel < ApplicationCable::Channel
  def subscribed
    stream_from "notifications:#{current_user_id}"
  end
end
```

Denne opsætning med RabbitMQ, Action Cable og Redis er særligt vigtig i en microservicearkitektur, hvor flere instanser af samme service kan køre parallelt, da den muliggør håndtering og distribution af stateful WebSocket forbindelser på tværs af instanser uden behov for direkte routing mellem dem. Arkitekturen bidrager til højere skalerbarhed og robusthed, da systemet kan håndtere stigende belastning uden at være afhængig af en enkelt serviceinstans, samtidig med at brugerne pålideligt modtager opdateringer i realtid.

3.1.3 DLS Patterns

Immutable data

Munchora anvender delvist principper fra immutable data gennem versionsstyring implementeret med Ruby gem'en PaperTrail. Entiteterne for Munchora såsom "recipe", "shopping_list" og "user" er mutable, men alle ændringer registreres som historiske snapshots i separate versionstabeller frem for at overskrive tidligere tilstande uden sporbarhed. PaperTrail integreres ved at tilføje til en given ActiveRecord model

"has_paper_trail", hvorefter gem'en automatisk registrerer ændringer ved brug af callbacks "create", "update" og "destroy". Der oprettes en versionstabel, hvor databasemodificeringer for en given model med "has_papertrail" gemmes som historiske versioner, hvilket sikrer fuld sporbarhed af dataenes livscyklus.

Denne tilgang sikrer historik over ændringer for at understøtte audit logging, rollback og sporbarhed i systemet. Løsningen kan perspektiveres til snapshot pattern (Tharpe u.d.), hvor data er immutable forstået som, at der kun må skrives til databasen og det er tidstemplet, som afgør, hvilken instans af dataen, der er den valide. Samtidig kan tilgangen også delvist minde om tombstone pattern, hvor slettet data fortsat kan spores historisk i et "user_deletes" table eller lignende frem for at blive permanent fjernet fra systemet. Dette bidrager til højere dataintegritet og gør det muligt at analysere ændringer på tværs af systemets levetid.

Asynchronous Messaging Pattern

Munchora anvender et "asynchronous messaging pattern" til håndtering af den asynkrone proces for AI generering af opskrifter. Når recipes-service anmoder om generering af en opskrift, sendes et event via RabbitMQ til ai-service, som i stedet for at udføre processen synkront direkte i request flowet håndteres dette asynkront ved hjælp af Sidekiq, som håndterer jobkøer og baggrundsprocessering. Denne tilgang sikrer, at AI generering ikke blokerer Rails serverens øvrige indkomne requests. Ved at flytte "tungere" operationer til baggrundsjob reduceres risikoen for overbelastning af servicen, samtidig med at systemets parathed og skalerbarhed forbedres. Kombinationen af RabbitMQ og Sidekiq gør det desuden muligt at afkoble services fra hinanden og behandle workloads mere robust og fleksibelt.

Service til service kommunikation

Kommunikationen mellem services er primært baseret på RabbitMQ og følger et publish/subscribe pattern, hvor services kan publicere og abonnere events igennem forskellige kanaler uden direkte kobling til hinanden. Denne tilgang bidrager til en mere robust og fejltolerant arkitektur sammenlignet med traditionel synkron HTTP kommunikation mellem services. Ved direkte http requests vil en utilgængelig service

kunne medføre fejl eller tab af funktionalitet i andre dele af systemet. Med RabbitMQ placeres beskeder derimod i køer, hvor de kan behandles, når den modtagende service igen er tilgængelig. Dette reducerer risikoen for tabte events og gør systemet mere modstandsdygtigt over for midlertidige nedbrud eller netværksfejl. Den asynkrone kommunikation gør det samtidig muligt at afkoble services fra hinanden, således at de enkelte services kan udvikles, deployes og skaleres uafhængigt. Dette understøtter microservice arkitekturens principper og bidrager til en mere fleksibel og skalerbar løsning.

3.1.4 Databases

Den primære anvendte database for Munchora er MySQL. Valget af en relationel database har jeg truffet på baggrund af databasens modenhed, stabilitet og understøttelse af ACID principperne og hvor det foretrækkes, at entiteterne er strukturerede efter normaliseringsprincipper. Ligeledes er data nemt at relatere til andet data igennem fremmede nøgler og mange-til-mange relationer kan etableres igennem en relations tabel såsom "shopping_list_shares". Dette sikrer høj konsistens og dataintegritet på tværs af systemets services. Hver microservice ejer sin egen database, hvilket understøtter løs kobling mellem services og reducerer direkte afhængigheder i arkitekturen. Denne tilgang medfører dog også forøget kompleksitet, da "user" entiteten eksisterer i auth-service, mens øvrige services såsom recipe-service og shopping-list-service fortsat har behov for information om, hvilken bruger der eksempelvis har oprettet en opskrift eller en indkøbsliste. Dette har jeg løst ved, at de enkelte services har deres egne domænespecifikke brugerrepræsentationer, såsom recipe_author og shopping_list_owner, som refererer til brugeren via attributten auth_user_id (se evt bilag 7.1 der viser den endelige implementering igennem fysiske entitets diagrammer).

Databaserne administreres af ActiveRecord migrations, hvilket giver en migrationsorienteret tilgang til ændringer i databasens struktur (DDL). Dette gør det muligt at versionere og reproducere databaseændringer på tværs af miljøer, som håndteres mere sikkert og kontrolleret gennem kode frem for manuelle ændringer direkte i databasen.

Indeksering anvendes aktivt for at optimere databasens læsetid. Dette gælder både almindelige indeks og composite indeks, hvor flere kolonner kombineres i samme indeks for at optimere mere komplekse queries. Et eksempel på dette ses i følgende ActiveRecord migration:

```
# munchora/backend/ai-service/db/migrate/20260115084949_add_indexes_to_llm_usages.rb
class AddIndexesToLlmUsages < ActiveRecord::Migration[8.0]
  def change
    add_index :llm_usages, :user_id
    add_index :llm_usages, [:user_id, :created_at]
  end
end
```

Her oprettes først et almindeligt indeks på `user_id`, som forbedrer performance ved opslag relateret til specifikke brugere. Der oprettes også et composite index på både `"user_id"` og `"created_at"`, for at understøtte tilhørende forretningslogik i `ai-service`, nemlig metoden `"usage_limit_exceeded?"`, som kontrollerer hvor mange prompts en bruger har foretaget inden for en given tidsperiode, begrænset til maksimalt 10 prompts om dagen. Der anvendes også Redis i dele af systemet til caching og pub/sub kommunikation. Redis benyttes i forbindelse med Sidekiq og ActionCable, hvor hurtig in-memory databehandling er nødvendig for at understøtte asynkron processering og realtidskommunikation.

Alternativt til en relationel database kunne jeg have brugt en dokumentbaseret database såsom MongoDB, særligt til mere fleksibel håndtering af semistruktureret data, hvilket kunne være recipe som har flere forskellige relationer til andet data og på sigt kunne udvides til at have endnu mere.

Derudover ville en grafbaseret database såsom Neo4j være relevant i relation til sociale funktioner for recipe entiteten som likes, kommentarer og relationer mellem brugere og opskrifter. En grafdatabase er designet til effektiv håndtering af komplekse relationer mellem noder og ville derfor kunne være fordelagtig i et system, hvor brugere interagerer intensivt med hinanden og med indhold. I en traditionel relationel database vil sådanne relationer ofte kræve mere komplekse joins og relationstabeller, hvilket kan påvirke kompleksitet og performance ved skalering. På nuværende tidspunkt har jeg vurderet MySQL til at være det mest hensigtsmæssige valg i forhold til projektets omfang og krav, mens alternative databasetyper potentielt kan være relevante ved videreudvikling og større skalering af platformen.

3.2 Environments

Munchoras produktionsmiljø består af en monolitisk udgave af Munchora, som er hostet på en virtuel maskine igennem Linode (Akamai Cloud), hvor jeg blot har anvendt en minimalistisk serverkonfiguration: 1 CPU-core, 1 GB RAM og 25 GB lagerplads. Denne opsætning er tilstrækkelig til applikationens nuværende behov og giver samtidig en omkostningseffektiv hostingløsning. Systemet er containeriseret med Docker ved, at separate images bygges til frontend og backend. Docker-images publiceres automatisk til GitHub Container Registry (GHCR) ved brug af GitHub Actions ".github/workflows/docker-registry.yml", som igangsættes, når kode skubbes ind i main branchen. Det har jeg valgt for at opnå en ensartet og reproducerbar proces uden at manuelt skulle bygge docker images ved kodeændringer.

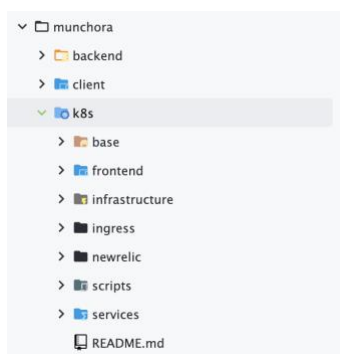
På serveren anvender jeg Docker Compose til at hente og starte de nødvendige containere, for at administrere hele systemet som en samlet enhed og forenkle deployment- og driftsprocessen. Munchora er eksponeret gennem en Nginx reverse proxy, som håndterer indkommende trafik på domænet <https://munchora.pro>. Proxyen videresender requests til den relevante container afhængigt af URL-strukturen. Requests med præfikset `/api/v1/*` sendes til Rails serveren, mens øvrige requests håndteres af frontend applikationen. På den måde er det muligt at afvikle både frontend og backend under samme domæne. Derudover har jeg konfigureret HTTPS med Let's Encrypt og Certbot, hvilket sikrer krypteret kommunikation mellem klient og server. Dette beskytter følsomme data såsom loginoplysninger og API requests mod eksempelvis man-in-the-middle angreb (Gregg Lindemulder u.d.). Ved yderligere skalering ville en mere distribueret infrastruktur baseret på Kubernetes og cloud-platforme som AWS, Azure eller Google Cloud være mere hensigtsmæssig og læne sig op ad den udviklede microservice orienterede arkitektur. En sådan løsning vil nemlig muliggøre automatisk skalering som giver en højere robusthed og forbedret opetid.

3.2.1 Udviklingsmiljø

Udviklingsmiljøet er baseret på Docker og Docker Compose for at sikre konsistente miljøer på tværs af services og udviklingsmaskiner. Denne opsætning muliggør hurtigere

opstart og konfiguration af systemet lokalt uden behovet for manuelt at skulle installere og konfigurere eksterne afhængigheder såsom databaser, forskellige backend services og nginx.

For at imødekomme den microservice orienterede tilgang har jeg også gjort brug af Kubernetes relaterede værktøjer såsom Minikube, kubectl og K9s med henblik på lokal orkestrering og administration af containere. Alle yaml filerne til opsætningen af Kubernetes såsom deployments, services og configmaps er at finde i projektet `munchora/k8s/`:



Figur 5 – munchora k8's fil struktur

K8s mappen indeholder desuden scripts til håndtering af Kubernetes ressourcerne, herunder oprettelse og fjernelse af secrets, configs og deployments. Disse scripts reducerer behovet for manuelle kubectl kommandoer og gør det lettere at administrere ændringer i pods, services og deployment konfigurationer. Formålet har været at emulere et så produktionslignende miljø som muligt for lokalt at kunne teste konfigurationen af de forskellige backend services, routing mellem services, integrationer til tredjepartssystemer såsom databaser, monitorering samt håndtering af secrets og miljøvariabler. Derudover har værktøjet K9s (se figur) været særligt nyttigt til at skabe overblik over de forskellige pods, deployments og services, som er spundet op i minikube.

Context: minikube [RW]	<0> all	<a> Attach	<ctrl-k> Kill	<0> Show Node
Cluster: minikube	<1> default	<ctrl-d> Delete	<l> Logs	<f> Show PortForward
User: minikube		<d> Describe	<p> Logs Previous	<t> Transfer
K9s Rev: v0.50.18		<e> Edit	<shift-f> Port-Forward	<w> Warp To Namespace
K8s Rev: v1.28.3		<?> Help	<z> Sanitize	<y> YAML
CPU: n/a		<shift-j> Jump Owner	<s> Shell	
MEM: n/a				

NAME	PF	READY	STATUS	RESTARTS	IP	NODE	AGE
ai-mysql-0	•	1/1	Running	3	10.244.1.83	minikube	39d
ai-rabbitmq-consumer-5969cff6d-tqwbq	•	1/1	Running	2	10.244.1.95	minikube	38d
ai-service-7d9dc849c8-lhw25	•	1/1	Running	3	10.244.1.97	minikube	38d
ai-sidekiq-worker-8459664f79-vtsm6	•	1/1	Running	2	10.244.1.99	minikube	38d
auth-mysql-0	•	1/1	Running	3	10.244.1.89	minikube	39d
auth-service-7c76878cd-xv7th	•	1/1	Running	4	10.244.1.87	minikube	38d
frontend-app-744b89586c-9mb86	•	1/1	Running	2	10.244.1.92	minikube	38d
notifications-rabbitmq-consumer-68d686948b-7f7g6	•	1/1	Running	2	10.244.1.80	minikube	38d
notifications-service-57d98854-ps2vj	•	1/1	Running	3	10.244.1.98	minikube	38d
rabbitmq-0	•	1/1	Running	3	10.244.1.78	minikube	39d
recipes-mysql-0	•	1/1	Running	3	10.244.1.77	minikube	39d
recipes-rabbitmq-consumer-8849b7f9d-jjxkd	•	1/1	Running	2	10.244.1.94	minikube	38d
recipes-service-5cbcb6f875-5hdsv	•	1/1	Running	4	10.244.1.86	minikube	38d
shopping-lists-mysql-0	•	1/1	Running	3	10.244.1.90	minikube	39d
shopping-lists-service-5d897d574c-j7wks	•	1/1	Running	4	10.244.1.85	minikube	38d

Figur 6 – k9s til overblik og konfiguration af lokalt minikube miljø

3.2.2 Monitorering

For at overvåge systemets stabilitet, performance og fejltilstande har jeg valgt at anvende New Relic til monitorering både af hver enkelt rails microservice og kubernetes clusteret.

Services - APM View all (5)					
	Name ↑	Response time	Throughput	Error rate	
☆	● munchora-ai-service	79 ms	7.67 rpm	0%	...
☆	● munchora-auth-service	90 ms	7.87 rpm	0%	...
☆	● munchora-notifications-service	59 ms	8.23 rpm	0%	...
☆	● munchora-recipes-service	87 ms	7.7 rpm	0%	...
☆	● munchora-shopping-lists-service	84 ms	8.57 rpm	0%	...

Figur 7 – monitorering af backend services med New Relic

New Relic tillader løbende at analysere svartider, resourceforbrug, fejl samt identificering af flaskehalse i systemet eksempelvis, hvis en service havde højere svartid end de andre. Jeg har valgt New Relic frem for en mere traditionel ELK-stack (Elasticsearch, Logstash og Kibana), fordi løsningen tilbyder en mere samlet og mindre driftskrævende platform. ELK stack'en ville kræve væsentligt mere opsætning, vedligeholdelse og infrastruktur, og jeg vurderede derfor, at New Relic ville være mest hensigtsmæssigt i forhold til projektets størrelse og tilgængelige ressourcer. Ved at anvende New Relic får man adgang til centraliseret logging, tracing af requests mellem services samt performance monitoring uden behov for selv at administrere den underliggende observability infrastruktur. Tracing

er relevant i en microservice arkitektur, hvor en enkelt brugerhandling kan involvere flere services og asynkrone processer.

4.0 Test

Munchora er opbygget som en mikroservicearkitektur, hvor funktionalitet er opdelt i selvstændige services, som bidrager til softwarekvaliteten ved at skabe klare ansvarsområder og reducere koblingen mellem forskellige dele af systemet. Det er muligt at udvikle, teste og deploye services uafhængigt af hinanden, hvilket gør koden mere overskuelig at vedligeholde og tillader en hurtigere udviklingsproces. Til gengæld introducerer arkitekturen mere kompleksitet, særligt i forhold til kommunikation mellem services, fejlfinding og håndtering af distribueret data. Netop derfor har implementeringen af tests under udviklingen af Munchora været en vigtig del af processen for at sikre stabilitet, korrekt funktionalitet og robusthed på tværs af de forskellige services. Test er en central del af softwareudvikling, men det er samtidig vigtigt at anvende det med omtanke og fokus på kvalitet frem for udelukkende kvantitet.

Munchoras backend services er udviklet i Ruby on Rails, hvortil jeg har valgt test frameworket RSpec fremfor det andet klassiske Ruby test framework MiniTest. RSpec er et veletableret og bredt anvendt værktøj i Ruby økosystemet, som understøtter både læsbare tests og en mere domæne og adfærds drevet tilgang sammenlignet med MiniTest, der er mere teknisk assertion drevet (Strzibny 2025).

Min tilgang til opsætning af testene er med en kombination af Detroit og London tilgang (Booth 2018), hvor der så vidt som muligt testes op imod reel implementering frem for udelukkende at anvende mocks. For en microservice arkitektur er det dog nødvendigt i visse tilfælde at mocke afhængigheder mellem services, for at sikre at hver service kan testes isoleret. Dette gør det også nemmere at afvikle tests i CI pipelinen, uden at have alle services kørende. Databasen er ikke mocket, men konfigureret som en Docker-baseret MySQL container "munchora/docker-compose.test.yml" for at sikre mere realistiske testscenarier.

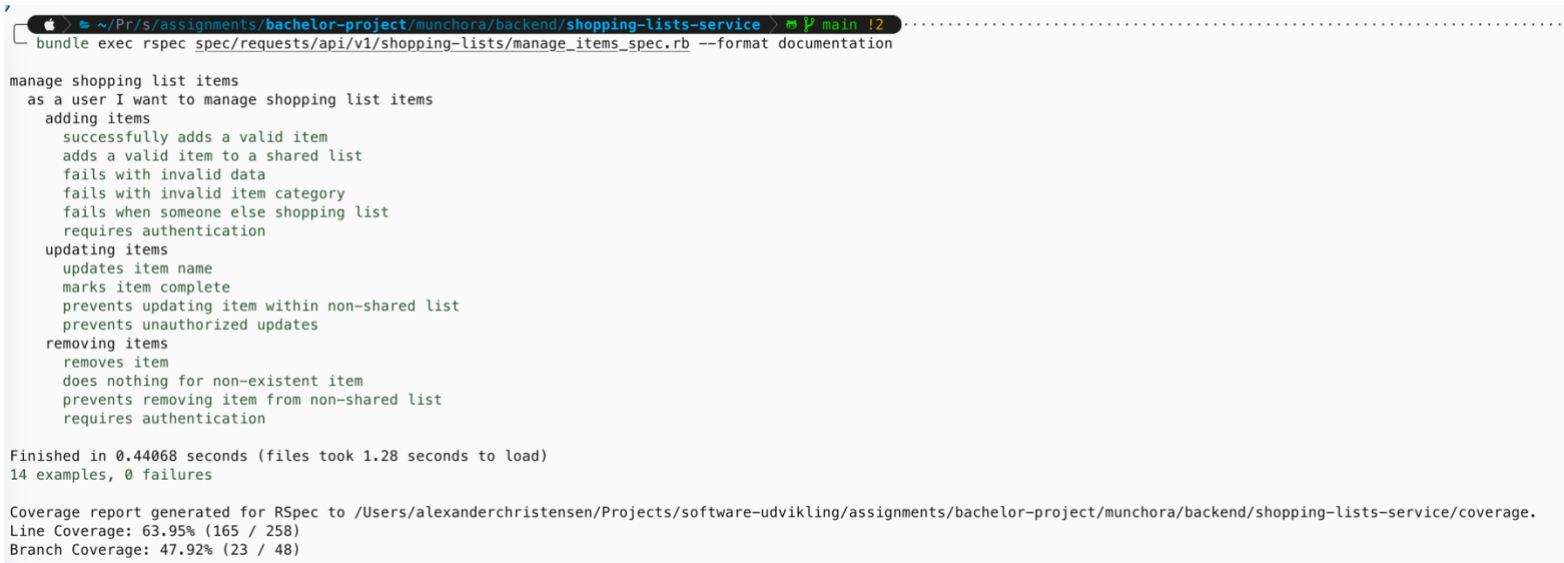
Jeg har lagt vægt på at udarbejde både positive og negative tests, for de mest kritiske og centrale funktioner i Munchora, såsom autentifikation, brugere, opskrifter og indkøbslister.

Jeg har ikke gået efter 100% test coverage, da dette ikke nødvendigvis altid er den mest hensigtsmæssige tilgang. Jeg har i højere grad forsøgt at fokusere på kvaliteten og relevansen af de tests, der er implementeret. En høj test coverage kan skabe en falsk tryghed, hvis kvaliteten af testene ikke er i orden, hvor koden derfor vil fremstå korrekt verificeret af de kørte succesfulde tests, selvom den i realiteten indeholder fejl (Draniceanu 2025).

I dette projekt er der lagt vægt på anvendelsen af BDD til udførelsen af tests, hvor de beskriver systemets forventede adfærd frem for udelukkende at verificere implementeringsdetaljer. Denne tilgang gør det muligt at anvende tests som en form for dokumentation, der giver indsigt i både positive og negative scenarier. Ved brug af RSpec's dokumentationsformat via kommandoen;

```
bundle exec rspec --format documentation
```

får man et overblik over systemets funktionalitet og forventede opførsel. Dette bidrager til en bedre forståelse af systemet og understøtter både udvikling og vedligeholdelse.



```
~/Pr/s/assignments/bachelor-project/munchora/backend/shopping-lists-service > main 12
bundle exec rspec spec/requests/api/v1/shopping-lists/manage_items_spec.rb --format documentation

manage shopping list items
  as a user I want to manage shopping list items
    adding items
      successfully adds a valid item
      adds a valid item to a shared list
      fails with invalid data
      fails with invalid item category
      fails when someone else shopping list
      requires authentication
    updating items
      updates item name
      marks item complete
      prevents updating item within non-shared list
      prevents unauthorized updates
    removing items
      removes item
      does nothing for non-existent item
      prevents removing item from non-shared list
      requires authentication

Finished in 0.44068 seconds (files took 1.28 seconds to load)
14 examples, 0 failures

Coverage report generated for RSpec to /Users/alexanderchristensen/Projects/software-udvikling/assignments/bachelor-project/munchora/backend/shopping-lists-service/coverage.
Line Coverage: 63.95% (165 / 258)
Branch Coverage: 47.92% (23 / 48)
```

Figur 8 - RSpec's dokumentationsformat, hvor tests skrevet efter BDD kan fungere, som en læsbar og meningsfuld dokumentation af kodebasens adfærd.

4.1 Unit Tests

Jeg har implementeret unit tests for de centrale dele af Munchora, hvilket er brugere, opskrifter og indkøbslister. Med disse tests har jeg ønsket at verificere om

domænelogikken fungerer for den enkelte service ved at isolere den fra resten af systemet. Dette bidrager samtidig til at identificere fejl tidligt i udviklingsprocessen. Disse domænemodeller har jeg håndteret ved brug af ActiveRecord, som definerer datastruktur, relationer og valideringer (ActiveModel). Dette gælder eksempelvis User modellen, som håndterer både manuel registrering og OAuth baseret autentificering via Google, samtidig med at modellen validerer felter, relationer og beskytter følsomme attributter ved serialisering af data. Testene er skrevet med fokus på både positive og negative scenarier for at afdække forskellige "paths" i systemet. Jeg har anvendt principper fra både black box og white box testing. Det er blandt andet inputvalidering, forretningslogik, relationer mellem modeller samt fejlscenarier og edge cases. Derudover har jeg anvendt 3-point boundary value analysis til tests, for at verificere hvordan systemet reagerer omkring grænseværdier såsom minimums- og maksimumslængder på inputfelter.

```

munchora/backend/auth-service/spec/models/user_spec.rb
RSpec.describe User, type: :model do
  describe 'validations' do
    let(:valid_attributes) do
      {
        email: "john@example.com",
        first_name: "John",
        last_name: "Doe",
        password: "secure123",
        provider: nil,
        uid: nil,
        bio: "Loves food.",
        image_src: "https://example.com/avatar.jpg"
      }
    end
    # =====
    # EMAIL VALIDATIONS
    # =====
    context 'email', :email_context do
      [
        # Invalid email partition 0 - 6: lower boundary
        { email: '', is_valid: false },
        { email: 't', is_valid: false }, # +1 char
        { email: 't.d', is_valid: false }, # equivalence partition
        { email: 'u@t.d', is_valid: false }, # -1 char from valid lower boundary

        # Valid email partition 6-100
        { email: 'u@t.dk', is_valid: true }, # valid lower
        { email: 'u1@t.dk', is_valid: true }, # +1 char
        { email: 'johnUser_232@test.dk', is_valid: true }, # equivalence partition
        { email: "#{ 'a' * 87 }@example.com", is_valid: true }, # -1 char from valid upper
        { email: "#{ 'a' * 88 }@example.com", is_valid: true }, # valid upper

        # Invalid email partition > 100
        { email: "#{ 'a' * 89 }@example.com", is_valid: false }, # +1 char
        { email: "#{ 'a' * 450 }@example.com", is_valid: false }, # equivalence partition

        # Edge cases: unexpected data type, wrong email format
        { email: nil, is_valid: false },
        { email: 1, is_valid: false },
        { email: true, is_valid: false },
        { email: 'plainaddress', is_valid: false },
        { email: '@missinguser.com', is_valid: false },
        { email: 'user@.com', is_valid: false },
        { email: 'user@com,.com', is_valid: false },
        { email: 'user@exa mple.com', is_valid: false }
      ].each do |example|
        email = example[:email]
        size_or_datatype = email.is_a?(String) ? "length #{email.size}" : email.class

        it "#{example[:is_valid]} ? 'accepts ' : 'rejects in' valid email with #{size_or_datatype}" do
          user = User.new(valid_attributes.merge(email: email))
          example[:is_valid] ? (expect(user).to be_valid) : (expect(user).to_not be_valid)
        end
      end
    end
    # Omitted code...
  end
end

```

Eksemplet her er et udpluk af unit tests tilhørende User-modellen, hvor der blandt andet ses anvendelse af 3-point boundary value analysis, equivalence partitioning samt tests af edge cases.

Jeg har fokuseret på høj test coverage og branch coverage omkring de mest kritiske funktioner i systemet for at sikre forskellige "paths" og betingelser bliver eksekveret og verificeret, eksempelvis er det kun en bruger, som må læse sin egen private opskrift og

blokering hvis en uautoriseret bruger forsøger det samme. Mit mål har dog ikke været blindt at opnå 100% coverage, men at skrive meningsfulde tests, som forsøger at skabe værdi ved at øge tilliden til systemets stabilitet og adfærd ved BDD-test af kernefunktionerne.

4.2 Integration og API Tests

Udover unit tests har jeg implementeret integration og API tests med fokus på systemets komponenter og deres endpoints/controllers. Formålet med disse tests har været at verificere, at services, requests, middleware og forretningslogik fungerer korrekt i samspil med hinanden, frem for udelukkende at blive testet isoleret på klasseniveau i den specifikke service. Jeg har arbejdet med integration tests omkring autentificering og autorisation i auth-service for at sikre, at JWT tokens både kan udstedes og valideres korrekt, samt at brugere kan registrere sig og korrekt autentificeres gennem systemet. Det er udelukkende auth-service, som har adgang til den private PEM nøgle, som signerer JWT tokens. Derfor har det været nødvendigt at mocke dele af token valideringen i øvrige services under test. På den måde har været muligt at teste services isoleret uden afhængighed til auth-service, samtidig med at autorisationsflowet fortsat kan simuleres realistisk i den pågældende service.

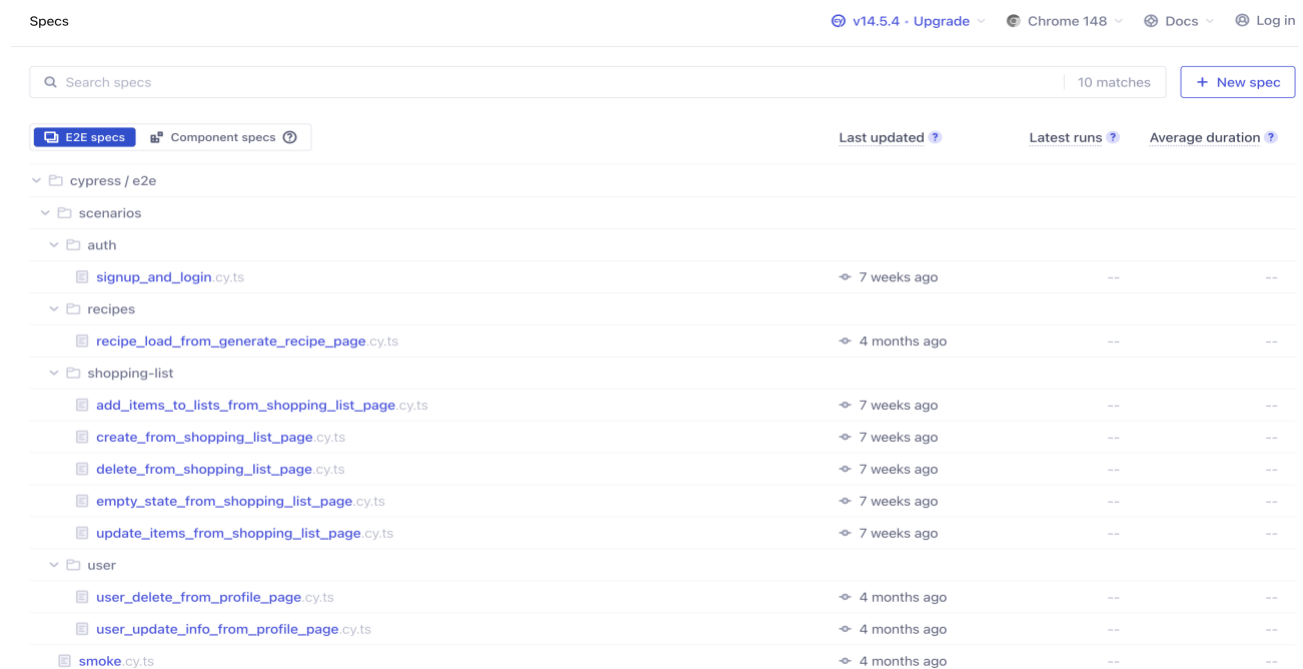
```
# munchora/backend/shopping-lists-service/spec/requests/api/v1/shopping-lists/manage_lists_spec.rb
RSpec.describe 'Manage shopping lists', type: :request do
  let(:user) { build(:current_user, user_id: 1) }
  let(:token) { "fake.jwt.token" }
  before do
    allow(Auth::JsonWebToken).to receive(:decode)
      .with(token)
      .and_return({ 'user' => HashWithIndifferentAccess.new(user.attributes) })
  end
  # omitted code...
end
```

Jeg har implementeret integration tests for shopping-list-service med fokus på de mest gængse brugerflows såsom håndtering af indkøbslister, deling af indkøbslister og administration af varer. Testene er skrevet ud fra en BDD inspireret tilgang, hvor specifikationerne i høj grad afspejler konkrete user stories og forventet brugeradfærd. Disse tests bidrager til, at overordnede dele af Munchoras API adfærd verificeres og sikrer, at centrale workflows fungerer korrekt på tværs af modeller, controllers, services og autorisationstilgange.

Ydermere kunne jeg have anvendt et værktøj såsom Postman til mere komplette automatiserede API tests med miljøvariabler, before/after scripts og delte collections. Postman giver nemlig mulighed for at eksportere oprettede collections som JSON og automatisk eksekvere komplette API test flows, hvilket er særligt relevant for strukturen i CI/CD pipelines. Som solo udvikler har jeg dog ikke prioriteret dette, da en sådan opsætning er tidskrævende i forhold til projektets omfang, men det kan være relevant ved en større skalering af Munchora.

4.3 End to end tests

Til end-to-end (E2E) testing har jeg anvendt Cypress fremfor Playwright eller Selenium. Det har jeg gjort på baggrund af dets intuitive udviklings oplevelse. Særligt Cypress' grafiske interface gør det nemt at afvikle tests kontinuerligt under udvikling, hvor tests automatisk geneksekveres ved ændringer i applikationen, hvilket gør fejlsøgningen og udviklingen væsentligt mere effektiv.



Specs		v14.5.4 · Upgrade	Chrome 148	Docs	Log in
Search specs		10 matches		+ New spec	
E2E specs		Component specs			
cypress / e2e		Last updated		Latest runs	
scenarios					
auth					
signup_and_login.cy.ts		7 weeks ago		--	
recipes					
recipe_load_from_generate_recipe_page.cy.ts		4 months ago		--	
shopping-list					
add_items_to_lists_from_shopping_list_page.cy.ts		7 weeks ago		--	
create_from_shopping_list_page.cy.ts		7 weeks ago		--	
delete_from_shopping_list_page.cy.ts		7 weeks ago		--	
empty_state_from_shopping_list_page.cy.ts		7 weeks ago		--	
update_items_from_shopping_list_page.cy.ts		7 weeks ago		--	
user					
user_delete_from_profile_page.cy.ts		4 months ago		--	
user_update_info_from_profile_page.cy.ts		4 months ago		--	
smoke.cy.ts		4 months ago		--	

Figur 8 – Cypress UI specs oversigt

Tilbage til starten af år 2000 har testpyramiden traditionelt dikteret, at størstedelen af tests burde være unit tests, efterfulgt af færre integrationstests og endnu færre E2E tests. Med moderne værktøjer som Cypress er denne tilgang dog i højere grad begyndt at ændre sig, da det i dag er blevet markant lettere og hurtigere at skrive og vedligeholde E2E tests, og man kan derfor vende test pyramiden på hovedet (Guerra 2024). E2E tests kan give stor værdi, da de tester hele systemets samlede flow fra brugerens perspektiv. Et opskrifts genererings flow vil eksempelvis involvere frontendens views, API kald til recipe-service og RabbitMQ kommunikation mellem recipe-, ai- og notifications-service. Hermed testes ikke blot enkelte komponenter isoleret, men hele applikationens samspil og forventede opførsel. Disse tests kan eksempelvis afvikles mod et staging miljø, hvor hele applikationen er deployet i et lukket og testsikkert miljø, som ikke påvirker produktionen. Det gør det muligt at validere systemets samlede funktionalitet under forhold, der minder om produktionsmiljøet, samtidig med at testene kan fungere som regressionstests. På den måde kan nye kodeændringer afprøves mod eksisterende funktionalitet for at reducere risikoen for utilsigtede "sideeffects" og introduktion af nye bugs. E2E tests bidrager derfor ikke kun til teknisk kvalitetssikring, men også til større sikkerhed omkring brugeroplevelsen og systemets forventede adfærd i praksis.

4.4 Continuous Integration (CI)

For at kvalitetssikre Munchora bliver den understøttet af en automatiseret CI-pipeline, som jeg har konfigureret ved brug af et Github Action workflow "Microservices - Continuous Testing", som kører parallelt for alle microservices ved enten push eller pull requests på main branchen. Workflowet er delt op i flere faser, hvor det væsentligste er, at der kører tests direkte i GitHub Actions workflowet. På den måde bliver hver ændring valideret før koden integreres i produktionsmiljøet, og risikoen for regressionsfejl formindskes, og det sikrer, at vedligeholdelsen af systemet forbliver overskuelig i takt med, at applikationen vokser. Workflowet er opdelt i flere jobs, som eksekveres parallelt for hver microservice. Først udføres linting med værktøjet RuboCop, som statisk analyserer Ruby koden og sikrer overholdelse af kodestandarder og best practices såsom korrekt indrykning, linjelængde og variabler der ikke er taget i brug. Herefter eksekveres

RSpec tests for hver microservice som regressionstests for at verificere, at de nye kodeændringer ikke introducerer fejl i den allerede eksisterende funktionalitet.

Mit "Continuous Testing" workflow er designet med matrix strategier, så hver service testes isoleret og uafhængigt af de øvrige services, hvilket passer godt til microservice arkitekturen, hvor services ideelt skal kunne udvikles, deployes og testes uafhængigt af hinanden. Derudover anvendes midlertidige MySQL containere direkte i workflowet, så testmiljøet så vidt muligt afspejler den reelle applikationsopsætning. Efter testeksekveringen for hver service uploades en artefakt i form af en coverage report, som kan gøre det nemmere at identificere områder af kodebasen med begrænset testdækning.

På nuværende tidspunkt er E2E tests ikke integreret i CI-pipelinen, men en mulig fremtidig løsning kan være automatisk at opstarte hele systemet ved Docker Compose, som E2E tests vil kunne eksekveres mod hele dette isolerede system.

Min anvendte "Continuous Testing" opsætning har hjulpet med at kvalitetssikre Munchora, da der løbende er blevet tjekket for korrekt linting og eksekvering af unit og integrationstests ved hver ændring i kodebasen. Dette har givet mig højere grad af tryghed og reduceret risikoen for regressionsfejl. Der er dog stadig potentiale for en mere komplet og helhedsorienteret teststrategi, særligt ved at udvide med E2E tests. En sådan udvidelse vil i fremtiden kunne give en endnu mere stabil og robust CI-oplevelse og dermed skabe større ro og sikkerhed for min videre udvikling af platformen.

5.0 Konklusion

Igennem mit forløb med at udvikle Munchora har jeg forsøgt at vise, hvordan en microservice orienteret arkitektur kan anvendes til at opbygge en skalerbar og modulær platform til AI opskriftsgenerering og indkøbshåndtering. De har været væsentligt for mig at designe services som stateless komponenter, hvor tilstanden i stedet håndteres eksternt. For at opnå dette har jeg gjort brug af Redis, som har været afgørende for håndteringen af både Sidekiq job queues og realtidskommunikation via ActionCable uden at holde på isoleret data i specifikke instanser af services. Jeg har valgt denne opsætning for at øge systemets fleksibilitet og på den måde muliggøre, at services horisontalt kan

skaleres uden risiko for tabte events eller andre sideeffekter, som kan opstå, når data opbevares lokalt i enkelte instanser af en service.

Jeg har valgt, at kommunikationen mellem services er håndteret gennem et event-baseret pub/sub system RabbitMQ, som bidrager til en mere robust og fejltolerant arkitektur i modsætning til direkte HTTP-kald services imellem. Simple HTTP-kald kan føre til tabte requests, eksempelvis hvis den anmodede service kortvarigt var nede, hvor RabbitMQ i stedet giver mulighed for asynkron behandling og bedre håndtering af midlertidige nedetider i enkelte services.

Herudover har jeg forsøgt at adskille domæner på tværs af services for at opnå en bedre maintainability. Ved at lade hver service eje sin egen database og sit eget domæne har det været muligt at reducere kobling og øge graden af uafhængighed i systemet. Det har gjort det lettere at udvikle, teste og vedligeholde de enkelte services isoleret samt reducere kompleksiteten i den samlede kodebase. Jeg har dog introduceret en øget arkitektonisk kompleksitet, da visse data må replikeres på tværs af services, såsom brugerentiteten, der oprindeligt lever i auth-service, men som også er nødvendig i blandt andet recipes-service og shopping-lists-service for at kunne knytte opskrifter og indkøbslister til brugeren. Dette har jeg håndteret ved at oprette separate domænespecifikke entiteter såsom RecipeAuthor og ShoppingListOwner, der refererer til brugeren via et auth_user_id. På den måde undgår jeg direkte databaseafhængigheder mellem services, samtidig med at de nødvendige forretningsmæssige relationer kan opretholdes. Samtidig har projektet også vist, at jeg har gået på kompromis med den valgte arkitektur. Selvom den relationelle datamodel med MySQL sikrer høj konsistens og stærke constraints, kunne alternative datamodeller have været relevante i visse domæner. Eksempelvis kunne en dokumentbaseret database som MongoDB have været fordelagtig til mere dynamiske entiteter såsom opskrifter, hvor strukturen potentielt kan ændre sig over tid og kræve mere fleksibel datarepræsentation. Samlet set vil jeg vove at konkludere, at arkitekturen for Munchora har skabt et fornuftigt fundament for en skalerbar platform, som dog vil kunne optimeres og forbedres. Særligt udvidelse af teststrategi, mere avanceret observability samt yderligere optimering af datahåndtering

og opsætning af fuldt kubernetes cluster i produktionsmiljøet vil kunne styrke systemets robusthed og driftssikkerhed for en fremtidig version.

6.0 Litteraturliste

- Booth, Adrian. 2018. "Test Driven Development Wars: Detroit vs London, Classicist vs Mockist." *Medium*. October 31. Accessed May 14, 2026. <https://medium.com/@adrianbooth/test-driven-development-wars-detroit-vs-london-classicist-vs-mockist-9956c78ae95f>.
- Cucumber. n.d. "Behaviour-Driven Development." *Cucumber*. Accessed May 13, 2026. <https://cucumber.io/docs/bdd/>.
- Draniceanu, Andreea. 2025. *Code Coverage: Why 100% Isn't the Holy Grail*. maj 22. Accessed april 15, 2026. <https://www.testim.io/blog/code-coverage-why-100-isnt-the-holy-grail/>.
- Fowler, Martin. 2014. *martinfowler*. March 25. Accessed March 22, 2026. <https://martinfowler.com/articles/microservices.html>.
- Gregg Lindemulder, Matthew Kosinski. n.d. "What is a man-in-the-middle (MITM) attack?" *IBM*. Accessed juni 05, 2026. <https://www.ibm.com/think/topics/man-in-the-middle>.
- Guerra, Gabriel. 2024. "The Testing Pyramid is upside-down ." *antithesis*. August 24. Accessed May 14, 2026. https://antithesis.com/blog/testing_pyramid/.
- Kosmowski, Błażej. 2017. "Essential RubyOnRails patterns - part 1: Service Objects." *Medium*. June 13. Accessed May 13, 2026. <https://medium.com/selleo/essential-rubyonrails-patterns-part-1-service-objects-1af9f9573ca1>.
- Noble Prog. 2017. *Requirements Management*. October 7. Accessed May 13, 2026. https://training-course-material.com/training/Requirements_Management.
- Radigan, Dan. n.d. "A guide to agile ceremonies and scrum meetings." *Atlassian*. Accessed May 13, 2026. <https://www.atlassian.com/agile/scrum/ceremonies>.
- Rails, Ruby on. n.d. *Action Cable Overview*. Accessed maj 7, 2026. https://guides.rubyonrails.org/action_cable_overview.html.
- . 2026. *Rails Guides*. Accessed marts 22, 2026. https://guides.rubyonrails.org/getting_started.html.

- Strmecki, Daniel. 2026. "Singleton Design Pattern vs Singleton Beans in Spring Boot ." *Baeldung*. March 18. Accessed May 13, 2026. <https://www.baeldung.com/spring-boot-singleton-vs-beans>.
- Strzibny, Josef. 2025. *Minitest vs RSpec for testing Rails applications*. marts 12. Accessed april 15, 2026. <https://testdrivingrails.com/blog/minitest-vs-rspec-for-testing-rails-applications>.
- Tharpe, James. n.d. *Snapshot design pattern* . Accessed maj 7, 2026. <https://www.jamestharpes.com/snapshot-pattern/>.

7.0 Bilag

7.1 DB Physical Entity Diagrams

7.1.1 ai_db

processed_prompts	
 correlation_id	varchar(255)
 created_at	datetime(6)
 updated_at	datetime(6)
 id	bigint

llm_usages	
 user_id	int
 provider	varchar(255)
 model	varchar(255)
 prompt	text
 prompt_tokens	int
 completion_tokens	int
 created_at	datetime(6)
 updated_at	datetime(6)
 id	bigint

ar_internal_metadata	
 value	varchar(255)
 created_at	datetime(6)
 updated_at	datetime(6)
 key	varchar(255)

schema_migrations	
 version	varchar(255)

7.1.2 auth_db

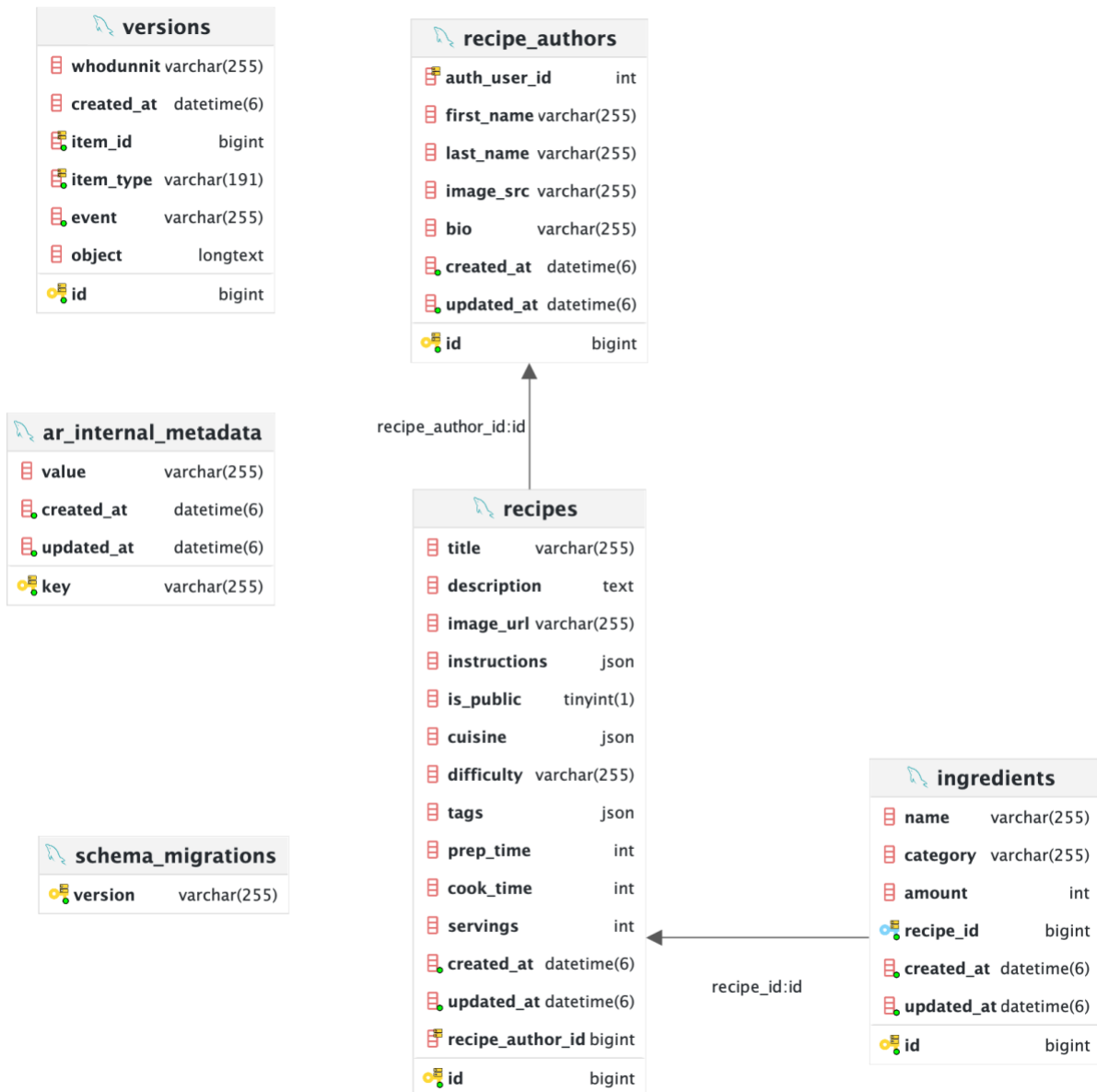
schema_migrations	
 version	varchar(255)

ar_internal_metadata	
 value	varchar(255)
 created_at	datetime(6)
 updated_at	datetime(6)
 key	varchar(255)

users	
 first_name	varchar(255)
 last_name	varchar(255)
 email	varchar(255)
 provider	varchar(255)
 uid	varchar(255)
 password_digest	varchar(255)
 image_src	varchar(255)
 bio	varchar(255)
 last_signed_in_at	datetime(6)
 created_at	datetime(6)
 updated_at	datetime(6)
 id	bigint

versions	
 whodunnit	varchar(255)
 created_at	datetime(6)
 item_id	bigint
 item_type	varchar(191)
 event	varchar(255)
 object	longtext
 id	bigint

7.1.3 recipes_db



7.1.4 shopping_lists_db

